AWS EKS Production Environment — Terraform + GitHub Actions + ArgoCD

Goal

Build a **production-grade Amazon Elastic Kubernetes Service (EKS)** platform on **AWS** to run a **microservices CRUD application** backed by **PostgreSQL**. The solution must be **fully automated end-to-end**: Terraform for infrastructure, GitHub Actions for CI, and ArgoCD (GitOps) for CD using Helm.

Branches (Separate Branches)

- **Application Branch:** `Deloitte-US/HU-DevOps-26-yourname`
- **Infrastructure Branch:** same one used for **IAC-DAY1**
- **CD (GitOps) Branch:** `<HU-DEVOPS-26-yourname>`

What You Need to Build

1) Infrastructure — Terraform Only

Provision everything with **Terraform** (no console clicking). Keep code **modular and DRY**. Use a **remote backend for state locking**. **Terraform plan/apply must run via pipeline**, not locally.

Must be created via Terraform

- **VPC** with:
  - **1 public subnet**
  - **2 private subnets**
  - **Required Components**
- **Private EC2 instance** (bastion/jump host) in a private subnet
- **EKS cluster**
  - **Private cluster** (not public-facing)

- Minimal footprint:
  - **2-node cluster**
  - **t2.medium** instances
  - Single node pool is fine

Notes / expectations

- Private networking should be designed so workloads can pull images and reach required endpoints without manual intervention.
- All required IAM (roles/policies), security groups, and cluster add-ons should be declared in Terraform (no one-off commands).

2) CI — GitHub Actions (Application Repo)

On every **push** (or **PR merge**), GitHub Actions must automatically:

- Build Docker images for **all microservices**
- Scan images (use **Trivy** or equivalent)
- Push images to **GCR (Google Container Registry)** (**NOT ECR**)
  - GCR prerequisites/setup must be included as part of **Terraform** in Step 1

Requirement: push code → pipeline runs → images are scanned and published to GCR with **no manual steps**.

3) CD — ArgoCD with Helm (GitOps)

Use **ArgoCD** for GitOps-style deployments. Every component must be deployed as a **Helm release** that ArgoCD continuously reconciles from the CD repo.

Services

- **Frontend** — serves UI and calls backend APIs
- **Users service** — CRUD for `users` table
- **Products service** — CRUD for `products` table
- **Orders service** — CRUD for `orders` table
- **PostgreSQL** — deployed in-cluster using the **Percona PostgreSQL Operator**

Important: All three backends connect to the **same Postgres instance** (single DB), each owning a different table.

4) Namespaces (Required)

Do not deploy into `default`. Use dedicated namespaces:

- `frontend` — frontend app
- `backend-users` — users service
- `backend-products` — products service
- `backend-orders` — orders service
- `database` — Percona operator + Postgres cluster
- `api-gateway` — Kong Gateway / Kubernetes Gateway API resources
- `monitoring` — Prometheus + Grafana + Tempo (observability core)
- `logging` — logging stack (**choose one: Loki OR ELK**, see below)
- `argocd` — ArgoCD
- `sealed-secrets` — Sealed Secrets controller
- `security` — Kyverno + related policies

Cross-namespace requirements

- Enforce least-privilege cross-namespace traffic using **NetworkPolicies**.
- Set appropriate **RBAC** for controllers/operators and GitOps reconciliation.
- API gateway must route to services across namespaces using Gateway API mechanisms.

5) Networking — API Gateway (Gateway API)

Do not use legacy Ingress. Use the **Kubernetes Gateway API** with a single external entrypoint.

- Deploy **Kong Gateway** (or another Gateway API-compatible implementation)
- **One** `LoadBalancer` only (single entry point)

Path-based routing (HTTPRoute)

- `/` → frontend (namespace: `frontend`)

- `/api/users` → users backend (`backend-users`)
- `/api/products` → products backend (`backend-products`)
- `/api/orders` → orders backend (`backend-orders`)

Required resources

- `GatewayClass`
- `Gateway` in `api-gateway`
- `HTTPRoute` resources (central or per-namespace—your choice)
- **ReferenceGrant** objects to allow cross-namespace backend references

6) Database — Percona PostgreSQL Operator (In-Cluster)

Deploy PostgreSQL inside Kubernetes using the **Percona PostgreSQL Operator** in the `database` namespace.

Requirements

- **Single Postgres instance**
- Single database with three tables:
  - `users`
  - `products`
  - `orders`
- Each backend service must only access its own table (enforced via app logic and least-privilege where feasible)
- **No hardcoded credentials**
  - Use **Sealed Secrets (kubeseal)** for all sensitive values (DB credentials, app secrets)

7) Observability (Metrics + Logs + Traces)

You must deliver end-to-end observability across infrastructure and applications.

Metrics (Prometheus + Grafana)

- Prometheus scrapes:
  - **EKS / Kubernetes metrics** (cluster/node/pod/namespace)

errors, etc.)

- Grafana dashboards must include at minimum:
  - API failure rates by service
  - HTTP request counts by service and endpoint
  - Response times: **P50, P95, P99**
  - CPU and memory usage **per pod** and **per namespace**
  - Key EKS/Kubernetes indicators (node readiness, pod restarts, deployment replica status, etc.)

Traces (Tempo)

- Distributed tracing with Tempo so a request can be followed:
  - gateway → frontend → backend → database interactions

Logs

Deploy **Loki** as the centralized logging stack in the `logging` namespace.

Requirements

- **Loki** for log aggregation (cluster-wide)
- A log collector/agent (e.g., **Promtail** or **Grafana Agent/Alloy**) to ship pod logs to Loki
- **Grafana** configured with **Loki as a datasource** and used as the log query UI (no ELK)

Expectations / demo proof

- Application logs from **frontend + all three backends + gateway + postgres/operator** are:
  - collected automatically
  - **searchable/queryable** in Grafana (Log Explorer)
- Logs are **structured** where possible and ideally **correlatable to traces** (e.g., trace/span IDs in log fields)
- Show at least:
  - filtering logs by **namespace/service/pod**
  - searching for **errors** and request IDs/trace IDs
  - time-range queries during the **node failure** test

Application instrumentation (required in all Flask services)

All Python (Flask) services must be instrumented to provide:

- **Traces** (OpenTelemetry)
- **Metrics** (exposed for scraping/collection)
- **Logs** (structured logs; correlated to trace IDs where possible)

Use OTEL components/collectors as needed so telemetry flows into:

- metrics → Prometheus/Grafana
- logs → Loki *or* ELK
- traces → Tempo

8) The Application (Python + Flask CRUD)

Build a simple microservices CRUD app.

Frontend

- Basic web UI dashboard to manage users/products/orders
- Proxies API calls to the backend services through the gateway routes

Backend services (each separate Flask app + Dockerfile + Helm chart)

- **Users service** (`users` table): `id`, `name`, `email`, `role`
- **Products service** (`products` table): `id`, `name`, `price`, `category`
- **Orders service** (`orders` table): `id`, `user_id`, `product_id`, `quantity`, `status`

9) Security

Requirements

- **No containers running as root**
  - Enforce with **Kyverno** (or OPA Gatekeeper) admission policy rejecting root execution

- **No secrets committed**
  - Use **Sealed Secrets** for sensitive configuration
- **Network isolation**
  - Default-deny NetworkPolicies; only allow required flows (gateway→services, services→db, telemetry shipping, DNS, etc.)
- **Branch protection**
  - Protect `main`; require PRs/reviews
- **Automation-first**
  - Entire platform should be deployable through pipelines without manual fixes

10) Service Mesh (Linkerd) — Zero-Trust Security & Validation

Objective

Enhance east–west application traffic security using Linkerd with **automatic mutual TLS (mTLS)** and **identity-based zero-trust access control**, scoped to application namespaces to fit the **2-node t2.medium** cluster.

Scope (mesh injection)

Included (meshed):

- `frontend`
- `backend-users`
- `backend-products`
- `backend-orders`

Excluded (non-meshed):

- `database`
- `monitoring`
- `logging`
- `argocd`
- `security`
- `sealed-secrets`
- `api-gateway`

Mesh Security Controls (required)

- **mTLS enforced** for all communication between meshed workloads.
- **Identity-based authorization** on meshed services so only explicitly allowed Linkerd identities can call them (least privilege).
- **NetworkPolicies remain enforced** (default-deny + explicit allows); Linkerd is an **additional** layer, not a replacement.

Mesh Validation / Demo Proof (required)

1. **mTLS proof (meshed → meshed):** Demonstrate that calls between meshed services (e.g., `frontend` → `backend-users`) are protected with mTLS.
2. **Zero-trust proof (non-meshed blocked):** From a pod in an excluded namespace (e.g., `database`), attempt to call a meshed backend service → **must fail.**
3. **Least-privilege identity proof (meshed denied/allowed):** Show that an allowed caller identity (e.g., `frontend`) can reach the intended backend, while another meshed identity that is not allowed (e.g., `backend-products` calling `backend-users`) is **denied.**
4. **Defense-in-depth confirmation:** Demonstrate NetworkPolicy enforcement still applies (mesh enabled does not bypass existing network isolation).

11) Resource Governance (ResourceQuota + LimitRange)

Requirements

- Implement **ResourceQuota** per namespace to cap total resource consumption (CPU/memory; optionally pods/services).
- Implement **LimitRange** per namespace to enforce sensible default **requests/limits.**
- Ensure every workload Helm chart sets explicit `resources.requests` and `resources.limits`.

"Use it wisely" expectations (2 × t2.medium)

- Quotas must be realistic so deployments schedule successfully and the **one-node-down test** can still run.

- Protect critical platform namespaces (`monitoring`, `logging`, `argocd`) from starvation.
- Document the quota rationale and show enforcement (e.g., a workload exceeding quota fails).

12) Autoscaling (KEDA) — Real-Time Scale Demo

Task:

Implement **KEDA (Kubernetes Event-driven Autoscaling)** to autoscale at least one stateless backend service (`backend-users` or `backend-products` or `backend-orders`) and demonstrate autoscaling **in real time** by generating load.

Requirements:

1. Deploy KEDA and create a **ScaledObject** for the chosen backend Deployment.
2. Set autoscaling bounds: **minReplicaCount** and a safe **maxReplicaCount** for a 2-node cluster.
3. Run a load tool/program to increase traffic to that backend API for several minutes and show:
    a. replicas **scale out** while load is high,
    b. replicas **scale in** after load stops (post-cooldown).
4. Provide evidence (screenshots or command output) of:
    a. ScaledObject trigger activation/status,
    b. Deployment replica count changes over time,
    c. Pods reaching Ready and serving requests.
5. Briefly explain how **ResourceQuota/LimitRange** could limit or block scale-out (e.g., Pending pods).

Acceptance criteria:

- Clear scale-out and scale-in behavior is demonstrated.
- Max replicas are capped to keep the cluster stable.
- Stateful components (e.g., Postgres) are **not** autoscaled.

13) Performance / Resilience Exercise (Node Failure Scenario)

You must demonstrate resilience behavior and how you manage a partial node outage.

Required test

- Bring down (or drain) **one node** in the 2-node cluster and demonstrate:
  - Workloads reschedule to the remaining node where capacity allows
  - Service remains available (or degrade is explained and measured)
  - ArgoCD continues to reconcile desired state
  - Observability shows the event (node not ready, pod rescheduling, error spikes if any)

Expected controls to discuss/demonstrate

- Replica strategy for stateless services (so one node loss doesn't take everything down)
- PodDisruptionBudgets (PDBs) and readiness/liveness probes
- Scheduling controls (anti-affinity/topology spread) where appropriate
- Clear explanation of database behavior under this constraint (single Postgres instance is a known availability limit)

What to Submit

A) Code

Your repos must include:

- **Terraform modules** (VPC, EKS, EC2, remote backend config, GCR prerequisites, etc.)
- **Helm charts** for:
  - frontend + 3 backends
  - Percona PostgreSQL Operator + Postgres cluster
  - api-gateway (Gateway API + implementation)
  - argocd

    - sealed-secrets
    - security (Kyverno + policies)
    - monitoring (Prometheus, Grafana, Tempo)
    - logging (**either** Loki stack **or** ELK stack) in `logging` namespace
- **GitHub Actions workflows**
    - CI: build + scan + push to GCR
    - Terraform: pipeline-driven plan/apply
- **Kyverno policies** + **NetworkPolicies** manifests

B) Documentation

Include documentation covering:

- Terraform structure (modules, environments, pipeline flow) and rationale
- Helm chart conventions and layout
- Namespace strategy and cross-namespace access model
- CI/CD flow end-to-end (where Trivy runs, how ArgoCD syncs)
- Security enforcement (admission control, network policies, sealed secrets)
- Observability wiring (metrics/logs/traces data paths and dashboards)
- Architecture diagram: frontend → 3 backends → Postgres (single instance; separate tables)